

L'agent et le firmware

Des pratiques agent-first vérifiées dans le dépôt
Omarchy — skills, treize harnesses, CLI lisible par
machine — jusqu'à la NVRAM : comment une couche
MCP et une HAL de sécurité donnent à l'agent une
prise réelle sur le firmware, sans jamais lui abandonner
le pouvoir de brique.

Ryzen 9 5950X · RTX 3070 · ASUS B450/B550

Omarchy 4.0 · MCP · fwupd · efivarfs · hwmon

Sommaire

1. Synthèse exécutive	1
1.1 Ce que ce volume démontre	1
1.2 La proposition en une page	2
2. Omarchy, un système agent-first	3
2.1 La preuve par le dépôt : AGENTS.md et ses guides	3
2.2 Treize harnesses, un lanceur, zéro fournisseur imposé	4
2.3 Les skills système : la discipline en lecture seule	5
3. Le fossé agent-firmware	6
3.1 Un agent aux mains liées devant le Setup	6
3.2 Pourquoi ce fossé n'est pas accidentel	7
3.3 Ce que l'agent ne peut pas faire aujourd'hui	7
4. Les canaux que Linux expose déjà	8
4.1 L'inventaire complet	8
4.2 fwupd et son bus Dbus : l'existant le plus proche d'une API firmware	9
4.3 Le contre-modèle des serveurs : Redfish, BMC et la leçon à retenir	9
5. MCP et Agent Skills : les standards de la couche agent	10
5.1 MCP : le protocole devenu standard de fait	10
5.2 Agent Skills : le format qu'Omarchy a déjà adopté	10
5.3 Le catalogue d'outils proposé	11
6. Architecture omarchy-firmware-agent	12
6.1 Les cinq couches, couche par couche	13
6.2 Cohérence avec les conventions du dépôt	14
6.3 Côté client : configurer les harnesses	14
7. Le modèle de sécurité : le firmware ne pardonne pas	15
7.1 Quatre tiers de risque, des verrous mécaniques	15
7.2 L'agent propose, la HAL dispose, l'humain tranche	17
7.3 Ce que le dépôt Omarchy enseigne au modèle de sécurité	17
8. Scénarios d'usage : ce que cela change	18
8.1 Un dialogue type, pas à pas	18
9. Prototype et feuille de route	19
9.1 L'outillage, en quelques fichiers	19
9.2 Métriques de succès et garde-fous de projet	20
10. Conclusion : l'agent change l'équation	21
10.1 Retour sur les trois trajectoires du volume 1	21
10.2 Le verdict	21
Sources	22

1. Synthèse exécutive

Le premier volume de cette étude s'était donné pour mission de cartographier le firmware AM4 et de proposer une suite « post-BIOS », **omarchy-firmware**, capable de déplacer vers Linux la quasi-totalité des usages du BIOS. Ce second volume part d'une intuition différente, exprimée par l'exploitation du système lui-même : Omarchy n'a pas seulement été construit *avec* des agents IA, il a été construit *pour* eux — et l'agent qui saurait parler au firmware serait l'aboutissement logique de cette trajectoire. L'hypothèse est séduisante ; ce rapport la prend au sérieux et la confronte aux faits, fichier par fichier, canal par canal, risque par risque.

La démonstration se déroule en trois temps. Le chapitre 2 établit d'abord la preuve matérielle : le dépôt Omarchy contient une infrastructure agent complète — un guide contributeur écrit pour les agents (**AGENTS.md** et ses sept guides de tâches), deux skills système livrés au format **SKILL.md**, treize lanceurs de harnesses pré-câblés avec leurs modes non-interactifs, un panneau de barre qui suit la consommation des abonnements IA, et un CLI explicitement documenté pour la lecture machine. Le chapitre 3 mesure ensuite l'écart : ce même agent, si puissant sur la couche OS, est totalement aveugle au firmware — ni lecture des réglages du Setup, ni connaissance de l'AGESA, ni prise sur les courbes de ventilation pré-OS. Les chapitres 4 à 6 construisent alors le pont : inventaire des canaux que Linux expose déjà, standards de la couche agent (MCP et Agent Skills), et architecture **omarchy-firmware-agent** qui soude le tout.

Le cœur de l'ouvrage reste le chapitre 7 : un agent autorisé à toucher au firmware est une idée excellente *et* dangereuse, et la différence entre les deux tient entièrement à l'architecture de sécurité. La proposition centrale tient en une phrase — **l'agent propose, la HAL dispose, l'humain tranche** — déclinée en quatre tiers de risque (T0 lecture, T1 configuration réversible, T2 NVRAM, T3 flash) dont les verrous sont mécaniques, jamais conversationnels, parce que l'injection de prompt documentée par l'OWASP peut toujours tromper un modèle, mais pas un verrou. Les chapitres 8 et 9 traduisent cette architecture en scénarios concrets et en feuille de route de huit semaines ; le chapitre 10 revient sur les trois trajectoires du volume 1, que l'agent-firmware recompose profondément.

13 harnesses agent pré-câblées dans Omarchy 4.0	2 standards mobilisés : MCP et Agent Skills	4 tiers de risque, verrous mécaniques T0 à T3	8 semaines pour un prototype en trois phases
--	---	---	--

1.1 Ce que ce volume démontre

Un — l'orientation agent d'Omarchy n'est pas un slogan, c'est une architecture vérifiable. Le fichier CLAUDE.md du dépôt se réduit à une ligne qui délègue à AGENTS.md ; ce dernier codifie le style bash, la nomenclature des commandes et les règles de privilège à destination des modèles. Le format SKILL.md —

frontmatter YAML puis instructions — est exactement celui que la fondation Agent Skills a publié comme standard ouvert en décembre 2025, ce qui fait d'Omarchy un adopteur précoce et conforme. Enfin, la commande **omarchy commands --json** expose les 455 commandes de la suite sous une forme directement consommable par un agent : la machine-lisibilité y est un objectif de conception, pas un accident.

Deux — le fossé agent-firmware est réel, mais il n'est pas infranchissable. Aucune API n'expose le Setup d'une carte B450/B550 : ni lecture des réglages courants, ni état de l'AGESA, ni courbes Q-Fan pré-OS. Mais Linux expose déjà dix canaux exploitables — SMBIOS, hwmon, nct6775, amd_pstate, MSR, efivarfs, le bus DBus de fwupd, les capsules UEFI, DDC/CI et le journal systemd — qui couvrent l'essentiel des usages, comme le montre le monde serveur où l'API Redfish du DMTF fait dialoguer les agents avec les BMC depuis des années. Le fossé est donc moins technologique qu'organisationnel : il manque la couche de médiation.

Trois — la sécurité n'est pas une option, c'est le produit. Écrire dans le firmware est la seule classe d'opérations capable de rendre une machine physiquement indisponible, et un agent peut être trompé par injection de prompt via le moindre contenu lu — page web, métadonnée de mise à jour, journal. Le volume 1 avait posé le blueprint de la suite ; ce volume y ajoute la HAL qui en constitue le sas : dry-run par défaut, profils de rollback, journal signé, confirmation humaine obligatoire au-delà de T1, interdiction absolue d'autonomie en T3. C'est cette discipline qui transforme l'intuition initiale en système digne de confiance.

1.2 La proposition en une page

omarchy-firmware-agent désigne la couche qui relie les treize harnesses d'Omarchy à la suite omarchy-firmware du volume 1. Concrètement, elle se compose de trois pièces : un **skill firmware** (SKILL.md) qui enseigne aux agents la prudence propre au domaine — la même discipline en lecture seule que le skill diagnose-crash applique aux crashes ; un **serveur MCP** qui expose huit outils typés (fw.audit.status, fw.boot.inspect, cpu.epp.set, fans.curve.set...) avec leur tier de risque déclaratif ; et la **CLI omarchy-firmware** elle-même, dont les métadonnées **# omarchy:requires-sudo=true** s'insèrent sans friction dans le routeur existant. Le tout passe par une **HAL unique** qui concentre les garde-fous : aucun chemin d'écriture ne l'évite.

Cette proposition ne réinvente rien — c'est précisément sa force. Elle réutilise le format de skill déjà déployé par Omarchy sur chaque machine, le protocole MCP devenu standard de facto depuis son adoption par OpenAI et Google en 2025, le bus DBus que fwupd publie déjà, et les conventions CLI du dépôt. Le lecteur pressé peut retenir l'équation suivante : la valeur du volume 1 (une suite post-BIOS complète) multipliée par la valeur de ce volume (une couche agent gouvernée) produit un système où le firmware cesse d'être un panneau caché pour devenir une API pilotée, auditée et journalisée — sans jamais déléguer à un modèle le pouvoir de brique.

2. Omarchy, un système agent-first

Avant de donner un agent au firmware, encore faut-il vérifier que le socle existe. Ce chapitre établit la preuve par le dépôt : chaque affirmation s'appuie sur un fichier daté du clone analysé (commit 5ead870, branche main, version 4.0.0.alpha). La conclusion est sans ambiguïté : Omarchy est la première distribution Linux grand public conçue avec les agents comme habitants de première classe, et cette conception laisse des traces concrètes dans presque tous les répertoires du projet.

2.1 La preuve par le dépôt : AGENTS.md et ses guides

Le signal le plus fort tient en une ligne. Le fichier **CLAUDE.md** — le nom que Claude Code cherche en entrant dans un projet — contient exactement « @AGENTS.md » : plutôt que d'entretenir une documentation parallèle pour chaque harness, Omarchy maintient un unique guide contributeur écrit *pour les agents*, que tous les outils incluent automatiquement. Ce document n'est pas un simple README : il codifie le style bash (bash 5, [[]], (())), la nomenclature des 450+ commandes, les règles d'escalade de privilège et la discipline de tests. Autrement dit, la charte d'ingénierie du projet est délivrée au modèle au même titre qu'à l'humain.

Sept guides de tâches complètent ce socle sous **agents/skills/**. Chacun encadre une catégorie de modification précise — ajouter une commande, écrire un script d'installation, modifier le shell Quickshell, créer une migration — avec les pièges connus et les tests à exécuter. C'est le format « procédure : fais ceci quand tu fais X », explicitement distingué dans AGENTS.md de la documentation de référence (docs/) et du manuel utilisateur (manual/). Cette division en trois arbres documentaires, pensée pour séparer auditoires, est une pratique de documentation agent-native rare même chez les éditeurs logiciels établis.

Guide (agents/skills/)	Ce qu'il encadre	Intérêt pour le modèle
command-metadata.md	Les métadonnées CLI (# omarchy:summary, :args, :requires-sudo...) scannées dans les 80 premières lignes	Le CLI devient self-describing ; l'agent découvre les commandes sans les exécuter
install-scripts.md	Les scripts sous install/ et les commandes de setup système	Évite les pièges de privilège et d'ordre d'exécution
shell-dev.md	Le shell Quickshell sous shell/	Cadre les modifications QML et leurs tests
icon-font.md	Les glyphes de marque du fichier omarchy.ttf	Procédure précise pour une ressource binaire
acceptance-tests.md	Les tests graphiques sous test/acceptance.d/ (VM jetable)	L'agent peut valider une UI sans casser la session
visual-verification.md	La vérification visuelle de tout changement d'interface	Compense l'absence de perception visuelle par une procédure
migrations.md	Les scripts de migration numérotés sous migrations/	Le modèle de mise à jour de l'OS devient authored-by-agent compatible

Table 2.1 — Les sept guides de tâches du dépôt, écrits pour les agents contributeurs.

2.2 Treize harnesses, un lanceur, zéro fournisseur imposé

Le chapitre AI du manuel décrit la politique d'Omarchy : traiter les agents de code comme des citoyens de première classe sans choisir de gagnant. Treize harnesses — Claude Code, Codex, OpenCode, Antigravity, Copilot CLI, Crush, Grok, Pi, Oh My Pi, Ori, Hermes, Muse et cursor-agent — sont pré-câblées comme des stubs paresseux gérés par mise : rien n'est téléchargé avant le premier appel, et **omarchy default agent** désigne celle qui répondra au raccourci Super+Shift+Ctrl+A. Le script **bin/omarchy-agent** révèle la profondeur de l'intégration : chaque harness reçoit son orthographe propre du mode « ne s'arrête pas pour demander » — `--permission-mode auto` pour Claude, `--approve-for-me` pour Codex, `--yolo` pour Crush ou cursor-agent, `--dangerously-skip-permissions` pour Antigravity.

Deux détails méritent d'être soulignés tant ils disent la maturité de l'approche. D'abord, un agent lancé par raccourci démarre dans `~/Work` et non dans `$HOME` : les harnesses refusant de mémoriser la confiance pour le répertoire personnel, Omarchy préfère déplacer la session que de la faire échouer. Ensuite, **omarchy agent prompt** « ... » permet de lancer un agent directement avec sa tâche — la commande de lancement est elle-même agent-callable, bouclant la boucle. Le panneau agents de la barre complète le tableau : il agrège l'usage des abonnements Claude, Codex et Fireworks (quota, jetons par jour et par modèle) régénéré toutes les quinze minutes, et se synchronise entre machines via un simple dossier partagé.

Harness	Mode non-interactif passé par omarchy-agent	Lecture
claude	--permission-mode auto	auto-approbation des outils
codex	--approve-for-me	approbation automatique
agy (Antigravity)	--dangerously-skip-permissions	aucune permission
copilot	--allow-all	toutes permissions
crush	--yolo	aucune invite
cursor-agent	--yolo --trust	commandes + confiance workspace
hermes	--yolo	exécution directe
muse	--approval-mode never	consigne d'approbation neutralisée

Table 2.2 — Extraits des modes non-interactifs codifiés dans bin/omarchy-agent (sélection représentative).

2.3 Les skills système : la discipline en lecture seule

Omarchy livre deux skills au format SKILL.md, installés par symlinks dans les répertoires standards de chaque harness (~/.claude/skills, ~/.codex/skills, ~/.pi/agent/skills, ~/.gemini/config/skills, ~/.hermes/skills et ~/.agents/skills). Le premier, **omarchy**, encadre la personnalisation du bureau : il interdit formellement de modifier /usr/share/omarchy (la lecture, elle, est encouragée), impose le pattern refresh-avec-sauvegarde et exige la confirmation de l'utilisateur avant tout reset. Ses règles d'escalade de privilège sont suffisamment précises pour être exécutables : **sudo quand un terminal est disponible pour le mot de passe, pkexec sinon** — notamment pour tout ce qu'un agent lance en arrière-plan.

Le second skill, **diagnose-crash**, est le plus instructif pour notre sujet. Branché sur la surveillance systemd-coredump, il est déclenché par une notification « Process crashed » et guide l'agent dans l'analyse du core dump : établir les faits, écarter les causes banales (OOM), corrélérer les horodatages, symboliser via debuginfod. Sa règle cardinale mérite d'être citée telle quelle : « *Leave the system as you found it. Diagnosis reads; it does not fix, tidy, or reconfigure.* » — laisse le système tel que tu l'as trouvé ; le diagnostic lit, il ne répare ni ne reconfigure. C'est la première pierre d'une éthique agent sur ce système : un périmètre de lecture assumé, des exceptions explicites et réversibles (le mute des notifications par programme), et l'honnêteté sur les limites de la preuve. Le skill firmware proposé au chapitre 6 transposera cette discipline mot pour mot.

Ce qu'il faut retenir. Omarchy a déjà résolu, pour la couche OS, les trois problèmes que pose l'agent-firmware : la découverte (métadonnées CLI lisibles par machine), l'habillage de sécurité (skills avec règles de privilège et périmètres en lecture seule) et l'observabilité (panneau d'usage, journal). La couche firmware n'a pas besoin d'inventer un modèle — elle a besoin d'appliquer celui qui fonctionne déjà.

3. Le fossé agent-firmware

Un agent Omarchy peut réécrire la configuration de Hyprland à chaud, diagnostiquer un core dump, gérer les paquets et même relancer le shell — mais il ne peut pas répondre à la question la plus simple qui soit : « quel est le réglage PBO actif sur ma machine ? ». Ce chapitre qualifie cet angle mort, le nomme — le **fossé agent-firmware** — et explique pourquoi il n'est pas un accident de l'histoire mais une conséquence structurelle de la conception des firmwares desktop.

3.1 Un agent aux mains liées devant le Setup

La surface sur laquelle l'agent agit aujourd'hui est celle que Linux expose à un utilisateur privilégié : fichiers de configuration, services systemd, paquets, journaux. C'est déjà immense, et le chapitre 2 a montré qu'Omarchy l'exploite méthodiquement. Mais la moitié de la machine échappe à cette carte : tout ce qui vit dans la puce SPI et se règle dans le Setup. Les réglages du processeur (PBO, Curve Optimizer, limites PPT), les profils mémoire (XMP/DOCP), les courbes de ventilation pré-OS, l'état du Secure Boot, la configuration du fTPM, l'ordre de boot profond — tout cela n'existe, pour le système en marche, que sous forme d'effets, jamais de causes lisibles.



Figure 3.1 — Le fossé : entre l'agent et la zone aveugle firmware, aucune API n'existe ; la surface runtime Linux, elle, est déjà entièrement accessible.

La figure 3.1 illustre l'asymétrie. Le tiers supérieur — l'agent et son terrain — et le bandeau central — la surface runtime — communiquent librement. Sous le bandeau LE FOSSE commence un domaine où l'agent ne lit rien, n'écrit rien et ne sait même pas ce qu'il ne sait pas. Un utilisateur humain franchit cette frontière sans y penser : il redémarre, appuie sur Suppr ou F2, navigue dans des menus. L'agent, lui, n'a ni écran ni clavier pré-OS ; le Setup est, par construction, une interface exclusivement humaine.

3.2 Pourquoi ce fossé n'est pas accidentel

Trois causes structurelles expliquent cet état de fait. La première est historique : le BIOS descend d'une époque où l'ordinateur n'avait pas de système d'exploitation à proprement parler ; son interface de configuration a été pensée pour un opérateur devant l'écran, et trente ans d'évolution UI n'ont pas changé ce postulat. La deuxième est économique : les constructeurs de cartes mères ne voient aucun marché dans une API de Setup — leur client cible règle XMP et les ventilateurs une fois, à l'installation. La troisième est sécuritaire : exposer le Setup via une interface programmable élargirait la surface d'attaque d'un composant déjà visé (LogoFAIL, Sinkclose), et la prudence des éditeurs de firmware les pousse plutôt vers le verrouillage.

Le contraste avec le monde serveur est éclairant. Là-bas, le BMC (Baseboard Management Controller) existe précisément pour offrir une prise programmatique sur la plateforme, hors système d'exploitation : l'API Redfish du DMTF permet d'interroger capteurs, inventaire et firmwares, de les mettre à jour, de contrôler l'alimentation — le tout en REST/JSON. Un serveur Supermicro, IBM (eBMC) ou NVIDIA DGX se gère aujourd'hui par des agents sans aucun obstacle de principe. Le desktop AM4 n'a simplement jamais reçu son équivalent ; le chapitre 4 montre que les canaux Linux existants en sont la compensation crédible.

3.3 Ce que l'agent ne peut pas faire aujourd'hui

Concrétisons l'angle mort par une liste de questions sans réponse sur une machine Omarchy actuelle. Chacune est banale pour un humain devant le Setup, inaccessible pour l'agent :

- **Lire** l'état des réglages du Setup : PBO activé ? Curve Optimizer négatif appliqué ? profil mémoire XMP/DOCP ? Limites PPT/TDC/EDC ?
- **Connaître** la version AGESA et le correctif qu'elle porte — l'agent doit recouper le BIOS rapporté par SMBIOS avec les notes de version du constructeur.
- **Ajuster** les courbes de ventilation pré-OS de la carte (Q-Fan ASUS) : seuls les contrôleurs exposés par hwmon (nct6775) sont atteignables, et leur mapping avec les courbes BIOS est approximatif.
- **Vérifier** l'atténuation effective d'une faille côté Setup (protection d'écriture SPI, verrou de logo LogoFAIL corrigé).
- **Comparer** deux profils firmware ou restaurer un profil — les profils Setup vivent dans la NVRAM, sans export scriptable côté vendor.

Cette liste définit précisément le cahier des charges négatif de notre architecture : tout ce qui n'y figure pas est déjà couvrable par la surface runtime, et tout ce qui y figure doit être soit contourné (profil, journal,

proxy de décision), soit explicitement laissé à l'humain avec l'agent en rôle de guide. C'est exactement la frontière que les quatre tiers du chapitre 7 formalisent.

4. Les canaux que Linux expose déjà

Franchir le fossé ne commence pas par une révolution du firmware mais par un inventaire honnête de ce que le noyau Linux met déjà à disposition. Ce chapitre recense dix canaux, évalue pour chacun la profondeur d'accès (lecture seule, écriture réversible, mise à jour) et identifie le module de la suite omarchy-firmware qui l'exploitera. Le message central est optimiste : la surface runtime couvre l'essentiel des usages quotidiens du BIOS, et la brique manquante est une couche de médiation, pas une nouvelle technologie.

4.1 L'inventaire complet

Canal	Ce qu'il expose	Accès	Module
SMBIOS / dmidecode	Carte, BIOS, version AGESA déclarée, mémoire, slots	Lecture	audit
sysfs hwmon	Températures, tachymètres ventilateurs, tensions, PWM	Lecture (+ écriture PWM)	fans
nct6775 (Nuvoton)	Contrôleur ventilateurs de la carte : courbes, sources, seuils	Lecture + écriture réversible	fans
amd_pstate / EPP	Gouverneur, préférence d'énergie (EPP), limites de fréquence	Lecture + écriture réversible	cpu
MSR / rAPL	Registres modèle-spécifiques, budget énergie PPL, compteurs	Lecture + écriture risquée	cpu
efivarfs / efibootmgr	Variables UEFI, entrées et ordre de boot, BootCurrent	Lecture + écriture NVRAM	boot
DBus org.freedesktop.fwupd	Inventaire des firmwares, catalogues, mises à jour signées	Lecture + installation	audit/update
UEFI capsule update	Dépôt d'une capsule signée traitée au prochain boot	Écriture critique	update
DDC/CI (i2c)	Réglages des écrans : luminosité, entrée, profil couleur	Lecture + écriture	audit
journal / coredumpctl	Historique boot, erreurs microcode, crashes corélés	Lecture	audit

Table 4.1 — Les dix canaux de la surface runtime Linux, avec profondeur d'accès et module exploitant.

4.2 fwupd et son bus DBus : l'existant le plus proche d'une API firmware

Parmi ces canaux, **fwupd** mérite un examen particulier car c'est le seul démon système d'Omarchy dont la mission explicite est le firmware. Son interface DBus (`org.freedesktop.fwupd`) expose une API propre : inventaire des périphériques updatables (`GetDevices`), catalogue des versions disponibles (`GetReleases`), installation de capsules signées vérifiées (`Install`), verrous d'état explicites — le démon refuse par exemple d'installer si la machine n'est pas branchée au secteur, et marque les mises à jour qui exigent un redémarrage. Le volume 1 a documenté sa limite majeure : le catalogue LVFS ignore la quasi-totalité des cartes mères desktop AM4, ASUS inclus.

Cette limite ne disqualifie pas fwupd pour autant — elle précise son rôle dans l'architecture. Pour les périphériques couverts (SSD, dock, éventuellement GPU), l'agent peut offrir le parcours complet : vérification, diff des notes de version, installation supervisée. Pour la carte mère elle-même, fwupd reste la source d'état (versions, drapeaux) et le modèle procédural à imiter : la procédure EZ Flash guidée du chapitre 8 reproduit ses garde-fous (secteur, checksum, confirmation) sans son automatisation. L'agent n'a pas besoin de contourner fwupd ; il a besoin de l'orchestrer.

4.3 Le contre-modèle des serveurs : Redfish, BMC et la leçon à retenir

Le monde serveur a résolu depuis une décennie le problème qui nous occupe. Chaque carte mère serveur embarque un microcontrôleur de gestion (BMC) qui survit aux redémarrages et expose **Redfish**, l'API REST normalisée par le DMTF : inventaire matériel, capteurs, journaux d'événements, mise à jour de firmwares (BMC, BIOS, CPLD), contrôle d'alimentation. Supermicro, HPE iLO, Dell iDRAC, IBM eBMC et les NVIDIA DGX la parlent nativement. Un agent moderne peut ainsi gérer le firmware d'un parc de serveurs sans jamais toucher à un clavier.

La leçon à en tirer est double. D'abord, l'idée d'un agent dialoguant avec le firmware n'a rien d'exotique : elle est la norme opérationnelle du datacenter, et les outils existants (Redfish, OpenBMC, ipmitool) en sont la preuve d'échelle. Ensuite, le desktop AM4 ne l'aura probablement jamais — aucun signe côté ASUS, MSI, Gigabyte ou ASRock d'un Redfish embarqué sur cartes grand public. La stratégie réaliste consiste donc à **construire l'équivalent logiciel côté OS** : la surface runtime du tableau 4.1 joue le rôle des capteurs et de l'inventaire, les capsules UEFI celui des mises à jour, et la HAL du chapitre 7 celui du contrôle d'accès. C'est un Redfish du pauvre — mais c'en est un, et il est gouvernable.

Frontière honnête. Ce que la surface runtime ne fera jamais : lire ou écrire les réglages internes du Setup (PBO, XMP, courbes pré-OS), ni flasher le BIOS sans boot spécial. Ce que la HAL peut garantir : que toute écriture possible l'est de façon réversible, journalisée et confirmée. Entre les deux, l'architecture choisit de documenter l'inaccessible plutôt que de le simuler.

5. MCP et Agent Skills : les standards de la couche agent

Entre les treize harnesses du chapitre 2 et les dix canaux du chapitre 4, il manque un langage commun. Deux standards récents le fournissent, et leur chronologie explique pourquoi 2026 est le bon moment : **MCP** (Model Context Protocol) pour le branchement d'outils, et la spécification **Agent Skills** pour la diffusion de procédures. Omarchy a déjà adopté le second ; ce chapitre montre que le premier s'y insère avec la même économie de moyens.

5.1 MCP : le protocole devenu standard de fait

Publié en open source par Anthropic en novembre 2024, le Model Context Protocol a été adopté par OpenAI puis Google dès le début de 2025, devenant le standard de facto pour brancher des outils sur des agents. Sa mécanique est simple : un serveur MCP — souvent un simple processus local parlant JSON-RPC sur stdio — expose des **outils** typés (nom, description, schéma d'arguments), que n'importe quelle harness compatible peut découvrir et appeler. La même découverte sert Claude Code, Codex, OpenCode, Hermes ou OpenClaw : écrire le serveur une fois, c'est équiper les treize harnesses d'Omarchy d'un coup.

Le précédent le plus proche existe déjà : le projet **dbus-mcp** expose automatiquement les services DBus d'une machine Linux comme outils MCP, avec introspection universelle. L'idée est séduisante mais insuffisante pour notre usage : le bus fwupd y serait présent, pas les subtilités du sysfs, et surtout aucun niveau de risque ni aucun garde-fou ne serait déclaré. Un serveur MCP *firmware dédié* reste donc nécessaire — mais il s'inspire du pont DBus pour la mécanique et de fwupd pour la discipline. Côté client, la configuration tient en quelques lignes de JSON dans chaque harness : le coût d'adoption est voisin de zéro.

5.2 Agent Skills : le format qu'Omarchy a déjà adopté

Introduites par Anthropic en octobre 2025 puis publiées comme spécification ouverte (agentskills.io) en décembre 2025, les **Agent Skills** sont des dossiers contenant un SKILL.md — frontmatter YAML (nom, description, déclencheurs) puis instructions Markdown — que les harnesses chargent quand leur domaine est sollicité. C'est exactement le format des deux skills système d'Omarchy décrits au chapitre 2, symlinkés dans les répertoires standards de chaque harness. La conformité au standard ouvert garantit la portabilité : un skill firmware écrit pour Omarchy fonctionnera tel quel sur toute harness conforme, y compris hors Omarchy.

La complémentarité des deux standards est totale et mérite d'être énoncée précisément. Le skill dit à l'agent **comment penser** le domaine firmware : la discipline en lecture d'abord, la règle du diagnostic qui ne répare pas, la cartographie des canaux, les signaux d'alerte (temps de boot dégradé, erreurs microcode dans le journal). Le serveur MCP lui donne **de quoi agir** : des outils typés avec périmètres déclarés, dont les écritures passent obligatoirement par la HAL. Un skill sans outils produit un agent qui conseille ; des outils sans skill produisent un agent qui improvise. Les deux ensemble produisent l'agent-firmware visé — et le monde a déjà validé ce duo dans d'autres domaines par la simple adoption.

```

---
name: firmware
description: >
Audit et gouvernance du firmware d'une machine Omarchy (AM4).
Utiliser pour l'état BIOS/AGESA, la chaîne de boot, fwupd, les
ventilateurs nct6775, l'EPP amd_pstate. LECTURE D'ABORD : toute
écriture passe par la HAL (tiers T1-T2, confirmation humaine).
Jamais de flash BIOS sans procédure humaine guidée (T3).
---
# Omarchy Firmware Skill
1. Ne jamais écrire avant d'avoir audité et proposé.
2. fw.audit.status est toujours le premier appel.
3. Toute écriture T1 : profil « avant » + dry-run + diff montré.
4. T2 (NVRAM) : confirmation humaine explicite, outil par outil.
5. T3 (flash) : produire le pas-à-pas humain, ne rien exécuter.

```

Le bloc ci-dessus esquisse le squelette du skill firmware — volontairement court, car un skill n'est pas une encyclopédie mais un déclencheur de discipline. Les règles qu'il porte reprennent mot pour mot l'éthique de diagnose-crash (lecture d'abord, système laissé en l'état) et la déclinent sur les quatre tiers formalisés au chapitre 7. Sa description YAML contient les déclencheurs lexicaux (« BIOS », « ventilateur », « PBO », « fwupd », « AGESA ») qui font que chaque harness la charge au bon moment, sans jamais l'imposer.

5.3 Le catalogue d'outils proposé

La table ci-dessous définit les huit outils du serveur MCP omarchy-firmware. Deux principes président à sa composition : chaque outil mappe sur une commande CLI existante ou planifiée du volume 1 (pas de duplication de logique), et chaque outil déclare son tier de risque — le serveur refusant structurellement tout appel hors périmètre plutôt que de le confier au bon vouloir du modèle.

Outil MCP	Action	Tier	Exemple de question couverte
fw.audit.status	Inventaire complet : carte, BIOS, AGESA, boot, fwupd	T0	« Où en est mon firmware ? »
fw.audit.cve	Croisement versions / CVE connues (vol. 1, ch. 5)	T0	« Suis-je exposé à LogoFAIL ? »
fw.boot.inspect	Chaîne UKI/Limine, entrées efibootmgr, snapshots Snapper	T0	« Ma chaîne de boot est-elle saine ? »
fw.update.check	fwupd GetReleases + catalogue LVFS, diff des notes	T0	« Y a-t-il des mises à jour ? »
cpu.epp.set	Préférence d'énergie EPP + gouverneur, profil avant/après	T1	« Optimise pour le gaming »
fans.curve.set	Courbes nct6775, dry-run puis application, rollback	T1	« Le ventirad hurle, calm-le »
fw.update.stage	Préparation capsule/fwupd avec verrous, reboot requis	T2	« Mets à jour le SSD en sécurité »
fw.rollback	Restauration profil NVRAM sauvegardé (efivars horodatés)	T2	« Reviens à l'ordre de boot d'hier »

Table 5.1 — Les huit outils du serveur MCP omarchy-firmware, leur tier de risque et leur usage.

6. Architecture omarchy-firmware-agent

Les pièces sont réunies ; ce chapitre les assemble. L'architecture suit une règle d'or héritée des skills d'Omarchy eux-mêmes : chaque couche ne parle qu'à ses voisines, et une seule frontière — la HAL — concentre l'autorité d'écriture. La figure 6.1 donne la vue d'ensemble ; les sections suivantes descendent couche par couche et vérifient la cohérence avec les conventions du dépôt.



Figure 6.1 — L'architecture en cinq couches : les treize harnesses parlent MCP, la CLI reste la référence, la HAL concentre les garde-fous, le noyau seul touche le firmware.

6.1 Les cinq couches, couche par couche

La **couche agent** est celle qui existe déjà : les treize harnesses avec leurs modes non-interactifs codifiés. Aucune modification n'y est nécessaire — c'est le bénéfice du choix MCP, qui rend le serveur consommable par toutes sans favoritisme. L'utilisateur continue d'invoquer **omarchy agent prompt** comme aujourd'hui ; seule la compétence disponible change. La **couche standards** est la nouveauté : le serveur MCP omarchy-firmware et le skill firmware de chapitre 5. Le serveur est un démon utilisateur léger (Python, stdio), le skill un dossier SKILL.md installé par les symlinks existants — les deux vivent dans le paquet omarchy-firmware et suivent ses mises à jour.

La **couche CLI** reprend la suite omarchy-firmware du volume 1 : cinq modules (audit, boot, cpu, fans, gpu) exposés au routeur omarchy avec les métadonnées standard. C'est le choix le plus structurant de l'architecture : la **CLI reste la seule référence fonctionnelle**, le serveur MCP n'étant qu'un habillage typé autour d'elle. Un humain garde ainsi l'accès direct à chaque fonction (omarchy firmware audit status), un script shell aussi, et l'agent utilise le chemin MCP — une seule implémentation, trois consommateurs. Les métadonnées # omarchy:requires-sudo=true marquent les commandes qui l'exigent, et la règle sudo/pkexec du skill omarchy s'applique telle quelle.

La couche HAL est le sas de sécurité décrit au chapitre 7 : elle intercepte toute opération d'écriture, applique le dry-run par défaut, maintient les profils de rollback, l'écriture du journal signé et l'exigence de confirmation humaine aux tiers supérieurs. **La couche noyau**, enfin, regroupe les interfaces sysfs, hwmon, DBus, efivarfs et MSR déjà inventoriées : elle n'est jamais atteinte directement par l'agent ni par le serveur MCP, mais uniquement par la HAL ou par les commandes CLI qui y passent. Cette règle de non-contournement est vérifiable par relecture de code — et testable en bloquant les permissions du groupe correspondant.

6.2 Cohérence avec les conventions du dépôt

L'architecture se vérifie enfin à l'aune des conventions d'Omarchy établies au chapitre 2. La découverte passe par **omarchy commands --json** : le groupe firmware apparaît automatiquement dans le listing machine-lisible dès que ses commandes portent les métadonnées standard — aucun patch du routeur. La sécurité repose sur les règles de privilège du skill omarchy (sudo avec terminal, pkexec sans) et sur l'interdiction absolue d'éditer /usr/share/omarchy : la suite s'installe sous /usr/share/omarchy/bin via le paquet, et les profils utilisateur sous ~/.config/omarchy/firmware/. La mise à jour suit le modèle migrations : un script daté peut évoluer le format des profils entre versions sans casser les machines.

Outil MCP	Commande CLI équivalente	Tier	Garde-fou HAL
fw.audit.status / cve	omarchy firmware audit status cve	T0	journal d'accès
fw.boot.inspect	omarchy firmware boot inspect	T0	journal d'accès
fw.update.check	omarchy firmware update check	T0	cache 15 min, pas de réseau sauf demande
cpu.epp.set	omarchy firmware cpu epp	T1	profil avant + dry-run + limites EPP codées
fans.curve.set	omarchy firmware fans curve set	T1	plancher tachymétrique + garde thermique + rollback
fw.update.stage	omarchy firmware update stage	T2	secteur requis + checksum + confirmation humaine
fw.rollback	omarchy firmware rollback	T2	confirmation humaine + vérification intégrité

Table 6.1 — La correspondance outils MCP / CLI / garde-fous : une seule implémentation, trois consommateurs.

6.3 Côté client : configurer les harnesses

L'expérience côté agent tient en une configuration de quelques lignes. Dans Claude Code, le serveur se déclare dans .mcp.json du projet ou dans la configuration utilisateur ; Codex, OpenCode et les autres suivent le même principe avec leur format propre. Hermes et OpenClaw, plus proches du système, peuvent lancer le serveur comme service utilisateur systemd (--user), ce qui permet le partage entre sessions. Le skill, lui,

n'exige aucune configuration : les symlinks de `default/agents/skills/` le rendent visible partout dès l'installation du paquet. À l'usage, l'utilisateur pose sa question en langage naturel — « pourquoi le ventirad hurle ? » — et la description YAML du skill oriente la harness vers l'outil adéquat ; le chapitre 8 déroule ces dialogues.

Principe d'unicité. Une fonction = une commande CLI = un outil MCP. L'agent n'est jamais un chemin de code parallèle : c'est un consommateur de plus de la même surface documentée. C'est ce qui distingue une architecture d'un prototype : la maintenabilité ne dépend pas de la présence de l'IA.

7. Le modèle de sécurité : le firmware ne pardonne pas

Tout ce qui précède serait irresponsable sans ce chapitre. Écrire dans la configuration du système est réversible par conception — snapshots Snapper, refresh avec sauvegarde, restaurations. Écrire dans le firmware ne jouit d'aucun de ces filets : une capsule maladroite ou une NVRAM corrompue peut rendre la machine incapable de démarrer, et la réparation exige alors un accès physique (clear CMOS, voire programmeur SPI). Ce chapitre érige donc la sécurité en architecture, pas en précaution.

7.1 Quatre tiers de risque, des verrous mécaniques

Le modèle retenu classe toute opération dans l'un de quatre tiers, du plus sûr au plus dangereux. Le principe directeur : **plus l'écriture est profonde, plus le pas de garde s'épaissit** — de l'accès libre et journalisé (T0) jusqu'à l'interdiction d'autonomie (T3). Ce qui distingue ce modèle d'une charte de bonne conduite, c'est la nature des verrous : ils sont **mécaniques**, codés dans la HAL, et non conversationnels. La raison est empirique : l'injection de prompt — classée LLM01:2025 par l'OWASP et décrite comme l'exploit IA le plus commun — permet de détourner un agent via le contenu qu'il lit ; un contenu hostile placé dans une note de version, un journal ou une page web peut tenter de lui faire franchir les lignes rouges. Un modèle peut être trompé ; un contrôle d'accès dans le code, non.

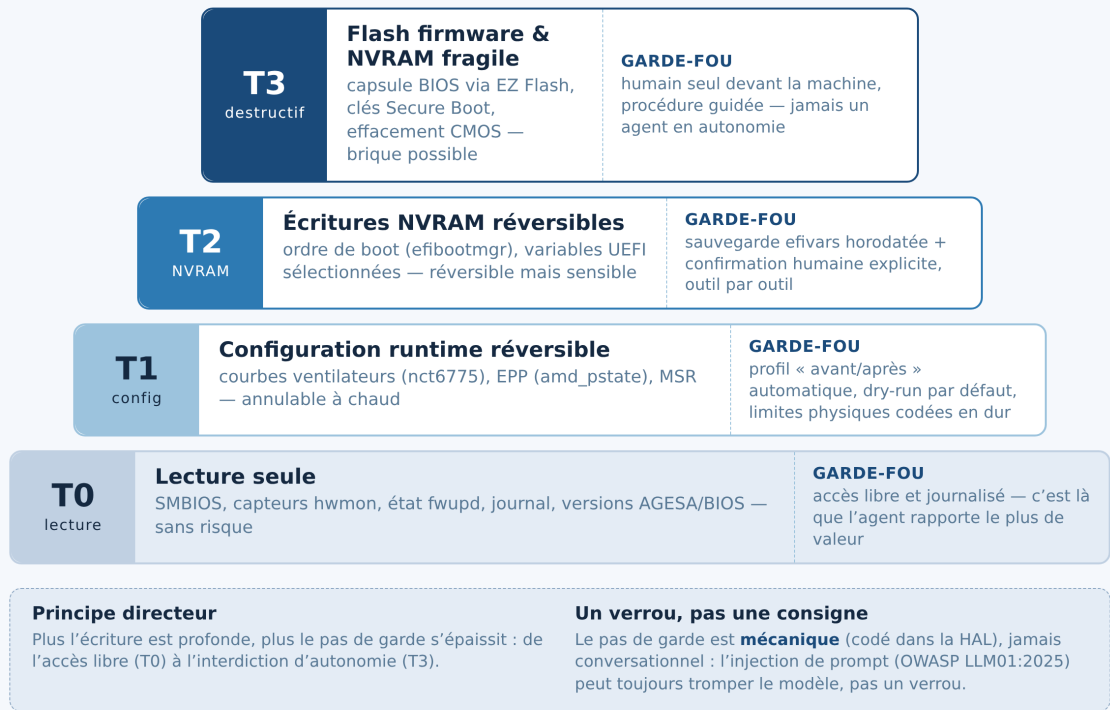


Figure 7.1 — La pyramide des quatre tiers : exemples d'opérations et pas de garde correspondant ; la note de bas rappelle que le verrou est mécanique, jamais conversationnel.

Menace	Vecteur	Mitigation dans l'architecture
Injection de prompt (OWASP LLM01:2025)	Contenu hostile lu par l'agent : note de version, journal, page web, courriel	Verrous HAL mécaniques ; le skill interdit d'agir sur instruction trouvée dans un contenu ; confirmation humaine hors T0
Capsule/métadonnée malveillante	Catalogue tiers, miroir compromis, précédent LogoFAIL (ressource BIOS malveillante)	Uniquement capsules signées vendor via fwupd/LVFS ; checksums vérifiés hors de l'agent ; aucune capsule fournie par l'agent
Corruption NVRAM / ordre de boot cassé	Écriture efivarfs hasardeuse, variable UEFI trop grande, double-boot Windows écrasé	Sauvegarde efivars horodatée avant chaque écriture T2 ; vérification BootOrder après ; refus si BootCurrent absent
Ventilation / puissance dangereuse	Courbe fans absurde, EPP extrême, MSR indiscret	Planchers tachymétriques et limites EPP codés en dur ; garde thermique ; interdiction MSR en écriture (lecture seule)
Flash BIOS interrompu	Coupure secteur pendant EZ Flash, capsule inadaptée	T3 hors autonomie : l'agent prépare et guide, l'humain exécute ; prérequis secteur + image signée constructeur

Table 7.1 — Le modèle de menaces de l'agent-firmware et ses mitigations.

7.2 L'agent propose, la HAL dispose, l'humain tranche

La formule résume la séparation des pouvoirs. L'**agent** dispose d'une initiative large : il audite, corrèle, propose, explique — c'est là que réside sa valeur. La **HAL** transforme l'initiative en acte selon des règles invariables : dry-run par défaut (l'agent voit le diff avant que quiconque ne l'approuve), profils « avant » automatiques, journal signé horodaté, limites physiques codées en dur. L'**humain** conserve le monopole des irréversibles : aucune écriture T2 sans confirmation explicite outil par outil, aucun passage T3 sans procédure guidée exécutée de ses mains. Ce tripartisme n'est pas une politesse envers l'utilisateur ; c'est la condition de l'assurabilité du système.

Il faut mesurer ce que le modèle refuse délibérément, car c'est là qu'il gagne sa crédibilité. Il refuse la maintenance entièrement autonome du firmware : pas de « mets à jour tout mon firmware pendant la nuit » sans revue. Il refuse l'optimisation blindée : pas de PBO agressif appliqué en silence parce qu'un benchmark l'a suggéré. Il refuse la réparation spontanée : face à un boot cassé, l'agent documente et propose, mais n'écrit pas dans la NVRAM sur sa propre inférence. En contrepartie, il offre une traçabilité que le BIOS n'a jamais eue : chaque écriture est datée, signée, reliée au profil de rollback qui l'annule — une avancée par rapport à l'ère où la seule trace d'un changement BIOS était la mémoire de celui qui l'avait tapé.

7.3 Ce que le dépôt Omarchy enseigne au modèle de sécurité

Ce modèle n'a rien d'importé : il généralise trois disciplines déjà présentes dans le dépôt. La première est la règle de diagnose-crash — le diagnostic lit et ne répare pas — que le tier T0 élargit à tout le domaine firmware. La deuxième est le pattern refresh-avec-sauvegarde du skill omarchy (backup horodaté avant toute restauration), que la HAL transpose en profils de rollback efivars/CMOS. La troisième est la ligne sudo/pkexec du même skill, qui devient la règle de privilège des commandes T1+. Même le panneau d'usage trouve son pendant : les écritures T1/T2 apparaissent dans une vue de journal consultable, donnant à la machine le même observabilité pour son firmware que pour ses agents. En somme, l'architecture n'ajoute qu'une pièce vraiment neuve — la HAL — et réutilise tout le reste.

La règle d'or, rappelée. L'agent propose, la HAL dispose, l'humain tranche. Un contenu hostile peut tromper l'agent ; il ne peut pas ouvrir la HAL. Un modèle capricieux peut proposer dix fois la même brique ; elle n'est exécutée ni dix fois, ni une seule, sans l'humain. C'est cette asymétrie — initiative large, exécution verrouillée — qui rend l'agent-firmware non seulement ingénieux, mais tenable.

8. Scénarios d'usage : ce que cela change

Une architecture se juge à ce qu'elle rend possible. Ce chapitre déroule cinq scénarios réalistes, choisis pour couvrir la gamme des tiers : trois de pure lecture (T0), une optimisation réversible (T1) et un passage guidé vers le T3. Pour chacun, on précise le prompt type, ce que l'agent fait réellement — outils appelés, canaux exploités — et le garde-fou engagé. L'objectif n'est pas de promettre la magie, mais de montrer que même sans accès au Setup, la valeur ajoutée est immédiate et mesurable.

Scénario	Prompt type	Ce que fait l'agent	Garde-fou
Ventirad bruyant	« Pourquoi le ventirad hurle en jeu ? »	Audit hwmon : températures vs tachymètres, courbe nct6775 active, EPP courant ; propose une courbe et un EPP ; montre le diff	T1 : profil avant + dry-run, planchers tachymétriques
Mise à jour sûre	« Mets le firmware à jour, sans surprise »	fw.update.check : versions LVFS, notes de version lues puis résumées ; carte mère absente du LVFS → produit le pas-à-pas EZ Flash daté et vérifié	T2 : verrous fwupd ; T3 : humain exécute, agent guide
Audit de boot	« Audite ma chaîne de démarrage »	fw.boot.inspect : UKI/Limine, entrées efibootmgr vs fichiers ESP, BootOrder, snapshots Snapper, protection LogoFAIL (version BIOS vs CVE)	T0 : journalisé, aucune écriture
Optimisation gaming	« Optimise la machine pour le gaming »	cpu.epp.set : EPP de balance à performance, gouverneur, vérification GSP côté GPU ; mesure avant/après via rAPL et fréquences	T1 : limites EPP codées, rollback en un appel
Veille CVE firmware	« Surveille les CVE et alerte-moi »	Boucle périodique (hook omarchy) : croise versions SMBIOS avec la base CVE du volume 1 ; notification omarchy-native si correspondance	T0 ; la boucle ne fait que lire et notifier

Table 8.1 — Cinq scénarios d'usage, du T0 au T3, avec garde-fou engagé.

8.1 Un dialogue type, pas à pas

Prenons le premier scénario et déroulons-le. L'utilisateur écrit : « le ventirad hurle quand je lance un jeu, règle ça sans que la machine chauffe ». L'agent commence par **fw.audit.status** (règle 2 du skill : l'audit d'abord), qui agrège carte, BIOS, capteurs et état des contrôleurs. Il observe alors le pattern classique : CPU à 78 °C avec un tachymètre bloqué à 600 tr/min jusqu'à 85 °C — la courbe Q-Fan du BIOS, réglée trop tolérante, puis le saut brutal à 100 % une fois le seuil franchi. Aucun mystère : c'est exactement le fossé du chapitre 3, la courbe pré-OS étant invisible, mais son *effet* est mesurable en continu par hwmon.

L'agent prépare alors deux actions T1 : une courbe nct6775 linéaire (40 % à 50 °C, 75 % à 80 °C) et un EPP performance pendant les sessions de jeu détectées. Il exécute les deux en dry-run, montre le diff — « la vitesse passera de 600 à 1 200 tr/min dès 70 °C » — et attend l'accord. À l'approbation, la HAL crée le profil

« avant » (l'état initial du contrôleur), applique, journalise. Deux semaines plus tard, l'utilisateur peut dire « reviens en arrière » et un seul appel fw.rollback restaure l'état d'origine, vérifié par la HAL. Le firmware vendor, lui, n'a pas bougé d'un octet : l'intégralité du parcours a eu lieu dans la surface runtime, gouvernée de bout en bout.

Ce que montre ce dialogue. L'agent n'a pas contourné le fossé — il l'a *comblé fonctionnellement* : la courbe BIOS reste inaccessible, mais l'équivalent runtime (nct6775) couvre le besoin, avec une gouvernance (profil, dry-run, journal, rollback) que le Setup n'offrira jamais. C'est la thèse du volume entier en un cas d'école.

9. Prototype et feuille de route

Le volume 1 avait posé une feuille de route en trois phases pour la suite omarchy-firmware ; celle-ci s'y insère sans la remplacer. La logique reste la même — la lecture d'abord, l'écriture gardée ensuite, la boucle supervisée enfin — mais chaque phase gagne un livrable agent : skill, serveur MCP, intégration panneau. Huit semaines suffisent à un développeur familier du dépôt pour livrer l'ensemble ; la table 9.1 fixe les critères de sortie objectivement vérifiables.

Phase	Semaines	Livrables	Critère de sortie
P1 — lecture seule	1-2	Skill firmware (SKILL.md) ; serveur MCP avec les 4 outils T0 ; CLI audit + boot inspect avec métadonnées ; journal d'accès	omarchy commands --json liste le groupe ; un agent pose 10 questions d'état sans aucune écriture
P2 — écritures gardées	3-5	HAL (dry-run, profils, rollback, journal signé) ; outils cpu.epp.set et fans.curve.set ; tests de non-contournement	100 % des écritures T1 passées par HAL ; rollback vérifié sur les deux modules ; refus prouvé des chemins directs
P3 — boucle supervisée	6-8	fw.update.check périodique (hook) + veille CVE ; staging T2 avec confirmations ; vue journal dans le panneau	5 jours de boucle sans faux positif ni écriture non confirmée ; pas-à-pas EZ Flash validé sur la carte cible

Table 9.1 — La feuille de route en trois phases : livrables et critères de sortie mesurables.

9.1 L'outillage, en quelques fichiers

Le périmètre de code reste délibérément minuscule au regard de la promesse. Côté CLI, chaque commande est un script bash de quelques dizaines de lignes, portant ses métadonnées en tête — exactement comme les 455 existants. Côté serveur MCP, un module Python unique enveloppe ces commandes : l'extrait ci-dessous

montre la définition de l'outil EPP, son tier déclaré et le passage par la HAL. Pas de framework, pas de démon root : un processus utilisateur et des scripts sudo listés, dans l'esprit du dépôt.

```
# bin/omarchy-firmware-cpu-epp (extrait des métadonnées)
# omarchy:summary=Set the CPU energy preference (EPP) profile
# omarchy:args=[balanced|performance|power-saver] [--dry-run]
# omarchy:requires-sudo=true
# serveur MCP (Python, extrait) – l'outil ne fait qu'envelopper la CLI
TOOL cpu_epp_set = {
  "name": "cpu.epp.set",
  "risk_tier": "T1",
  "requires": "hal.run('omarchy-firmware-cpu-epp', profile, dry_run=True)",
  "rollback": "hal.rollback_last()",
}
```

9.2 Métriques de succès et garde-fous de projet

Un prototype d'agent-firmware se juge sur des métriques négatives autant que positives. Côté positif : l'audit complet de boot en moins d'une minute, dix questions d'état types répondues sans intervention humaine, chaque écriture T1 annulée en un appel. Côté négatif — les plus importantes : zéro écriture T2 sans confirmation enregistrée, zéro accès T3 déclenché par un agent, zéro écriture hors HAL détectée par les tests de non-contournement. Ces métriques s'inscrivent dans le runner de tests existant (test/shell.d), pour que la sécurité soit une régression vérifiée comme une autre. Enfin, une règle de projet : aucun outil MCP n'entre dans le serveur sans son tier déclaré et son test de refus hors périmètre — la table 5.1 est un contrat, pas un catalogue d'intentions.

- **Temps d'audit boot** < 60 s de l'appel fw.boot.inspect au rapport complet (mesuré sur la machine cible).
- **Réversibilité T1** : 100 % des écritures ventilateurs/EPP annulables par fw.rollback, vérifié après chaque application.
- **Intégrité T2** : aucune écriture NVRAM sans profil « avant » horodaté correspondant dans ~/.config/omarchy/firmware/.
- **Autonomie T3** : zéro exécution possible — le serveur MCP n'expose même pas de chemin d'appel pour le flash.
- **Non-contournement** : les tests bloquent l'accès direct au groupe hwmon et prouvent que la CLI seule échoue proprement.

10. Conclusion : l'agent change l'équation

L'intuition de départ — un agent qui communique avec le BIOS serait peut-être ingénieux — ressort de cette étude confirmée, précisée et gardée. Confirmée, parce qu'Omarchy est réellement agent-first jusqu'à la moelle : guide contributeur unique, skills au standard ouvert, treize harnesses équipées, CLI machine-lisible. Précisée, parce que « parler au BIOS » signifie en réalité dialoguer avec la surface runtime que Linux expose — dix canaux, un démon fwupd, aucune API Setup — et que c'est largement suffisant pour couvrir l'usage. Gardée, enfin, parce que l'ingéniosité tient moins au dialogue qu'à sa gouvernance : l'agent propose, la HAL dispose, l'humain tranche.

10.1 Retour sur les trois trajectoires du volume 1

La **trajectoire A** — garder la carte ASUS et construire la suite post-BIOS — sort renforcée de ce volume. L'agent-firmware lui apporte ce qui lui manquait pour être convaincante auprès du plus grand nombre : une interface conversationnelle qui rend la suite accessible sans documentation, et une gouvernance qui la rend défendable. La **trajectoire B** (ASRock Rack X470D4U sous coreboot, BMC intégré) acquiert un nouveau sens : le BMC parle Redfish, donc l'agent y trouve nativement l'API que le desktop lui refuse — l'architecture MCP de ce volume pourrait d'ailleurs exposer Redfish comme outils additionnels sans rien changer d'autre. La **trajectoire C** (AM5 + Dasharo) reste l'horizon de long terme ; Dasharo publiant des firmwares ouverts, l'agent y gagnerait enfin une lecture profonde du Setup — l'objet même du fossé du chapitre 3.

10.2 Le verdict

Pour la machine étudiée — Ryzen 9 5950X, RTX 3070, carte ASUS B450/B550 sous Omarchy — la recommandation du volume 1 est réaffirmée avec un argument de plus : **construire omarchy-firmware-agent sur la trajectoire A**. Les trois phases de huit semaines du chapitre 9 livrent d'abord un auditeur imbattable (T0), puis un régleur réversible et journalisé (T1), enfin une sentinelle de mise à jour et de CVE (T2, avec l'humain aux commandes du T3). À chaque étape, la valeur est réelle et le risque contenu ; à aucun moment le système ne dépend d'une promesse des constructeurs.

La formulation finale de la thèse tient en trois propositions. **Un** : le firmware reste le dernier système fermé de la machine personnelle, non par nécessité technique mais par inertie — et Linux a déjà les clefs de la plupart de ses fonctions. **Deux** : les agents IA sont le consommateur naturel de ces clefs, à condition que l'architecture leur donne l'initiative sans l'exécution — c'est le rôle de la HAL et de ses verrous mécaniques. **Trois** : Omarchy est l'endroit où construire cela, parce qu'il a déjà résolu les problèmes adjacents — découverte, privilèges, observabilité, discipline de lecture — et que sa communauté parle déjà la langue des agents. Le BIOS a régné trente ans sur la partie invisible de la machine ; il reste une année de travail pour la rendre visible, gouvernée et conversationnelle.

Sources

Références principales consultées pour ce volume (accès vérifié en septembre 2026).

Dépôt basecamp/omarchy, commit 5ead870, v4.0.0.alpha : AGENTS.md ; CLAUDE.md ; agents/skills/ (7 guides) ; default/agents/skills/{omarchy,diagnose-crash}/SKILL.md ; bin/omarchy-agent ; bin/omarchy-default-agent ; shell/plugins/agents/ (README, collecteurs) ; docs/command-metadata ; manual/17-ai.md.

Anthropic — « Equipping agents for the real world with Agent Skills », octobre 2025 ; spécification Agent Skills, agentskills.io (publication ouverte, décembre 2025).

Anthropic — « Introducing the Model Context Protocol », novembre 2024 ; adoptions OpenAI et Google début 2025 (annonces publiques).

Spec MCP : outils, ressources, transports stdio/HTTP ; documentation des serveurs (modelcontextprotocol.io).

Projet dbus-mcp — pont D-Bus vers MCP avec découverte automatique des services (GitHub, 2025-2026).

fwupd — documentation développeur, interface D-Bus org.freedesktop.fwupd (GetDevices, GetReleases, Install) ; fwupd.org.

DMTF — « Redfish : API RESTful de gestion de plateforme » ; introductions Redfish de Supermicro, IBM (eBMC) et NVIDIA DGX (documentation constructeurs).

OWASP Gen AI Security Project — « LLM01:2025 Prompt Injection », top 10 des risques applicatifs LLM, 2025 ; analyses Obsidian Security (août 2026).

Arch Wiki — efivarfs, efibootmgr, dmidecode ; documentation noyau Linux : sysfs hwmon, amd_pstate, nct6775 (Documentation/hwmon).

Volume 1 de la présente étude — « Au-delà du BIOS : étude firmware Omarchy » (audit vendors AM4, CVE 2022-2026, blueprint omarchy-firmware), septembre 2026.